Universidad de San Buenaventura Sede Bogotá

Facultad de Ingeniería – Programas de Tecnologías

Trueque Market

Proyecto Integrador

Integrantes:

Alejandro Gutiérrez

Brayan Fontecha

Hernan Rojas

Bogotá, Colombia

2026

Tabla de Contenido

Tabla de Contenido 2
1. Introducción 4
2. Definición del Proyecto 4
2.1 Problemática 4
2.2 Justificación 4
3. Objetivos 5
3.1 Objetivo General 5
3.2 Objetivos Específicos 5
4. Gestión del Proyecto 5
4.1 Metodología Scrum Aplicada 5
Sprint 1 – Análisis y Planeación 5
Sprint 2 – Base de Datos y Autenticación 5
Sprint 3 – Desarrollo de Funcionalidades 5
Sprint 4 – Integración y Seguridad 6
Sprint 5 – Despliegue y Pruebas 6
4.2 Cronograma del Proyecto 6
5. Tecnologías Utilizadas 6
6. Arquitectura del Sistema 7
6.1 Frontend 7
6.2 Backend 7
6.3 Base de Datos 7
6.4 Hosting y Despliegue 7
6.5 Patrón Arquitectónico MTV 7
7. Framework Django 7
8. APIs Integradas 8
8.1 API Pusher 8
8.2 API OpenStreetMap 8
8.3 API MetaMap 8
9. Seguridad del Sistema 8
10. Gestión de Base de Datos 8
10.1 CRUD del Sistema 8
10.2 Comparación SQL y NoSQL 8
10.3 Propiedades ACID y BASE 9
10.4 Teorema CAP 9
11. Modelo Relacional del Sistema 9
11.1 Entidades Principales 9
11.2 Relaciones Implementadas 9

12. Estructuras Dinámicas Utilizadas 10
13. Diagramas UML 11
13.1 Diagrama de Estados 11
13.2 Diagrama de Clases 11
13.3 Diagrama de Actividades 12
13.4 Diagrama de Casos de Uso 13
14. Casos de Uso 15
14.1 Registro e Inicio de Sesión 15
Flujo principal 15
14.2 Crear Publicación 15
Flujo principal 15
14.3 Enviar Solicitud de Trueque 15
Flujo principal 15
14.4 Gestionar Trueque 15
14.5 Calificar Usuarios 15
15. Pruebas del Sistema 15
15.1 Resultados Obtenidos 16
16. Rendimiento y Escalabilidad 16
16.1 Resultados Observados 16
17. Conclusiones 17
Referencias 17

1. Introducción

El presente documento expone el desarrollo del proyecto integrador Trueque Market, una plataforma web orientada al intercambio de productos y servicios entre usuarios mediante herramientas digitales modernas.

El sistema fue diseñado con el propósito de promover la economía colaborativa, permitiendo que los usuarios puedan publicar artículos, gestionar solicitudes de intercambio, comunicarse entre sí y calificar experiencias de trueque dentro de una plataforma segura y organizada.

Durante el desarrollo se integraron conocimientos adquiridos en diferentes áreas académicas como desarrollo web, arquitectura de software, bases de datos relacionales, integración de APIs, seguridad informática y metodologías ágiles Scrum. La plataforma fue desarrollada utilizando tecnologías modernas como Django Framework, PostgreSQL, Python, HTML, CSS y Render, además de APIs externas para comunicación en tiempo real, validación de identidad y geolocalización.

2. Definición del Proyecto

Trueque Market consiste en una plataforma web orientada al intercambio de productos y servicios entre usuarios mediante acuerdos colaborativos. El sistema permite el registro de usuarios, autenticación, publicación de productos, envío de solicitudes de trueque, mensajería entre usuarios, calificaciones y gestión de intercambios.

La plataforma fue diseñada como una solución tecnológica enfocada en facilitar procesos de intercambio de manera digital y segura.

2.1 Problemática

Actualmente muchas personas poseen objetos que ya no utilizan pero que aún tienen valor para otras personas. Sin embargo, no existen suficientes plataformas accesibles enfocadas exclusivamente en el intercambio de productos mediante trueque.

Además, muchos procesos de intercambio informal presentan problemas relacionados con falta de confianza, poca organización, ausencia de validación de usuarios, dificultad de comunicación y falta de geolocalización. Por esta razón surge Trueque Market como una solución tecnológica enfocada en optimizar y organizar los procesos de intercambio mediante una plataforma digital moderna.

2.2 Justificación

El desarrollo de Trueque Market permite aplicar conocimientos adquiridos durante el semestre en diferentes áreas tecnológicas y de ingeniería de software. El proyecto integra desarrollo backend, desarrollo frontend, diseño de bases de datos, arquitectura de software, APIs, despliegue en la nube y seguridad informática.

Adicionalmente, el sistema busca promover la economía colaborativa y el aprovechamiento responsable de recursos mediante herramientas digitales modernas.

3. Objetivos

3.1 Objetivo General

Desarrollar una plataforma web funcional llamada Trueque Market que permita realizar intercambios de productos y servicios entre usuarios de manera segura, organizada y eficiente mediante herramientas modernas de desarrollo de software.

3.2 Objetivos Específicos

- Diseñar una interfaz web intuitiva para facilitar la interacción de los usuarios con el sistema.
- Implementar un sistema de autenticación y gestión de usuarios mediante Django Framework.
- Desarrollar un sistema CRUD para administrar publicaciones y solicitudes de intercambio.
- Implementar una base de datos PostgreSQL para garantizar integridad y persistencia de los datos.
- Integrar APIs externas para mejorar las funcionalidades del sistema.
- Aplicar principios de seguridad informática para proteger la información de los usuarios.
- Desplegar la aplicación en la nube mediante Render para garantizar disponibilidad remota.
- Aplicar metodologías ágiles Scrum para organizar el desarrollo del proyecto.

4. Gestión del Proyecto

Para el desarrollo de Trueque Market se implementó la metodología ágil Scrum, permitiendo organizar las diferentes fases del proyecto mediante sprints de trabajo. La metodología Scrum permitió organizar tareas, distribuir responsabilidades, controlar avances, corregir errores y optimizar el desarrollo.

4.1 Metodología Scrum Aplicada

Sprint 1 – Análisis y Planeación

- Análisis de requerimientos.
- Diseño inicial del sistema.
- Planeación de arquitectura.

Sprint 2 – Base de Datos y Autenticación

- Diseño de base de datos PostgreSQL.
- Implementación del sistema de autenticación.
- Registro e inicio de sesión.

Sprint 3 – Desarrollo de Funcionalidades

- Desarrollo del CRUD de publicaciones.

- Gestión de solicitudes de intercambio.
- Sistema de mensajes.

Sprint 4 – Integración y Seguridad

- Integración de APIs.
- Mejoras visuales.
- Seguridad del sistema y validaciones.

Sprint 5 – Despliegue y Pruebas

- Despliegue en Render.
- Pruebas funcionales.
- Corrección de errores y optimización del sistema.

4.2 Cronograma del Proyecto

Fase	Actividad	Duración
Planeación	Levantamiento de requerimientos	1 semana
Diseño	Arquitectura y base de datos	1 semana
Desarrollo	Frontend y Backend	3 semanas
Integración	APIs y PostgreSQL	1 semana
Pruebas	Correcciones y validaciones	1 semana
Despliegue	Publicación en Render	1 semana
Documentación	Informe final	1 semana

5. Tecnologías Utilizadas

Tecnología	Función
Python	Desarrollo backend
Django Framework	Arquitectura y lógica del sistema
PostgreSQL	Base de datos relacional
HTML	Estructura visual de la interfaz
CSS	Diseño y estilos visuales
Render	Hosting y despliegue en la nube
Pusher	Comunicación en tiempo real
OpenStreetMap	Geolocalización de usuarios
MetaMap	Validación de identidad

6. Arquitectura del Sistema

La arquitectura del sistema fue diseñada mediante una estructura modular basada en capas, permitiendo separación entre lógica de negocio, presentación y datos.

6.1 Frontend

Desarrollado mediante HTML y CSS. Sus funciones incluyen la interfaz gráfica, formularios, visualización de publicaciones e interacción con usuarios.

6.2 Backend

El backend fue desarrollado utilizando Django Framework. Sus funciones comprenden autenticación, lógica de negocio, validaciones, gestión de publicaciones, conexión con PostgreSQL, administración de usuarios e integración con APIs.

6.3 Base de Datos

Implementada mediante PostgreSQL. Sus funciones comprenden almacenamiento de usuarios, gestión de productos, mensajes, trueques y calificaciones.

6.4 Hosting y Despliegue

La plataforma fue desplegada mediante Render (https://render.com/), permitiendo acceso remoto y disponibilidad continua. La URL de acceso es: https://trueque-market-app-1-0-1.onrender.com/

6.5 Patrón Arquitectónico MTV

El sistema implementa el patrón Model-Template-View (MTV) propio de Django:

- Model: gestiona la conexión con PostgreSQL y los datos del sistema.
- Template: controla la interfaz visual mostrada al usuario.
- View: gestiona la lógica de negocio y las solicitudes del sistema.

Este patrón permitió mantener separación entre lógica, datos e interfaz gráfica, facilitando el mantenimiento y escalabilidad del proyecto.

7. Framework Django

Trueque Market implementa Django como framework principal de desarrollo backend. Entre sus principales ventajas aplicadas se encuentran:

- Arquitectura MTV nativa.
- Sistema de autenticación integrado.
- Protección CSRF incorporada.
- Manejo eficiente de rutas y URLs.
- Integración sencilla con PostgreSQL.
- Panel de administración automático mediante Django Admin.

8. APIs Integradas

8.1 API Pusher

Funciones: mensajería en tiempo real, sincronización de eventos y notificaciones. Beneficios: comunicación dinámica y mejor experiencia de usuario dentro de la plataforma.

8.2 API OpenStreetMap

Funciones: geolocalización, renderización de mapas y ubicación de usuarios. Beneficios: optimización logística y coordinación de intercambios físicos entre usuarios.

8.3 API MetaMap

Funciones: validación de identidad y verificación de usuarios. Beneficios: mayor seguridad y prevención de fraude dentro del sistema.

9. Seguridad del Sistema

Trueque Market implementa mecanismos de seguridad orientados a proteger la información de los usuarios:

- Autenticación de usuarios con sesiones seguras.
- Validación de formularios en frontend y backend.
- Protección de rutas privadas mediante decoradores Django.
- Sanitización de entradas para prevención de inyecciones.
- Protección CSRF mediante middleware de Django.
- Manejo seguro de credenciales mediante variables de entorno.

10. Gestión de Base de Datos

La plataforma implementa PostgreSQL como sistema gestor de base de datos relacional. PostgreSQL fue seleccionado debido a su alta confiabilidad, integridad relacional, estabilidad, soporte ACID y rendimiento en consultas estructuradas.

10.1 CRUD del Sistema

El sistema implementa operaciones CRUD para gestionar la información dentro de la plataforma:

- Create: permite registrar nuevas publicaciones de productos.
- Read: permite visualizar productos disponibles para intercambio.
- Update: permite modificar publicaciones existentes.
- Delete: permite eliminar publicaciones del sistema.

10.2 Comparación SQL y NoSQL

Característica	PostgreSQL	MongoDB
Tipo	Relacional	No Relacional
Modelo	Tablas	Documentos JSON
Consistencia	ACID	BASE
Escalabilidad	Vertical	Horizontal
Flexibilidad	Media	Alta
Integridad Relacional	Alta	Media
Uso ideal	Datos estructurados	Datos dinámicos

PostgreSQL fue seleccionado como base de datos principal debido a la necesidad de mantener integridad relacional y consistencia en las transacciones del sistema. MongoDB fue analizado como alternativa NoSQL por su flexibilidad y capacidad de escalabilidad en aplicaciones modernas.

10.3 Propiedades ACID y BASE

PostgreSQL implementa propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), garantizando seguridad y estabilidad en las transacciones. MongoDB implementa propiedades BASE (Basically Available, Soft State, Eventually Consistent), que permiten mayor flexibilidad en sistemas distribuidos.

10.4 Teorema CAP

El Teorema CAP establece que un sistema distribuido no puede garantizar simultáneamente Consistencia, Disponibilidad y Tolerancia a particiones. PostgreSQL prioriza consistencia e integridad de datos, lo cual resulta fundamental para garantizar estabilidad en Trueque Market.

11. Modelo Relacional del Sistema

El modelo entidad-relación fue diseñado para representar las interacciones dentro de la plataforma.

11.1 Entidades Principales

- User: datos de acceso y perfil del usuario.
- Perfil: información extendida del usuario.
- Producto: publicaciones disponibles para intercambio.
- Mensaje: comunicación entre usuarios.
- Trueque: registro de intercambios solicitados y realizados.
- Calificación: evaluaciones post-intercambio entre usuarios.

11.2 Relaciones Implementadas

- Un usuario puede publicar múltiples productos.

- Un usuario puede enviar y recibir mensajes.
- Un usuario puede participar en múltiples trueques.
- Los usuarios pueden calificarse mutuamente tras un intercambio.

El diseño relacional garantiza integridad, consistencia, normalización y eficiencia de consultas.

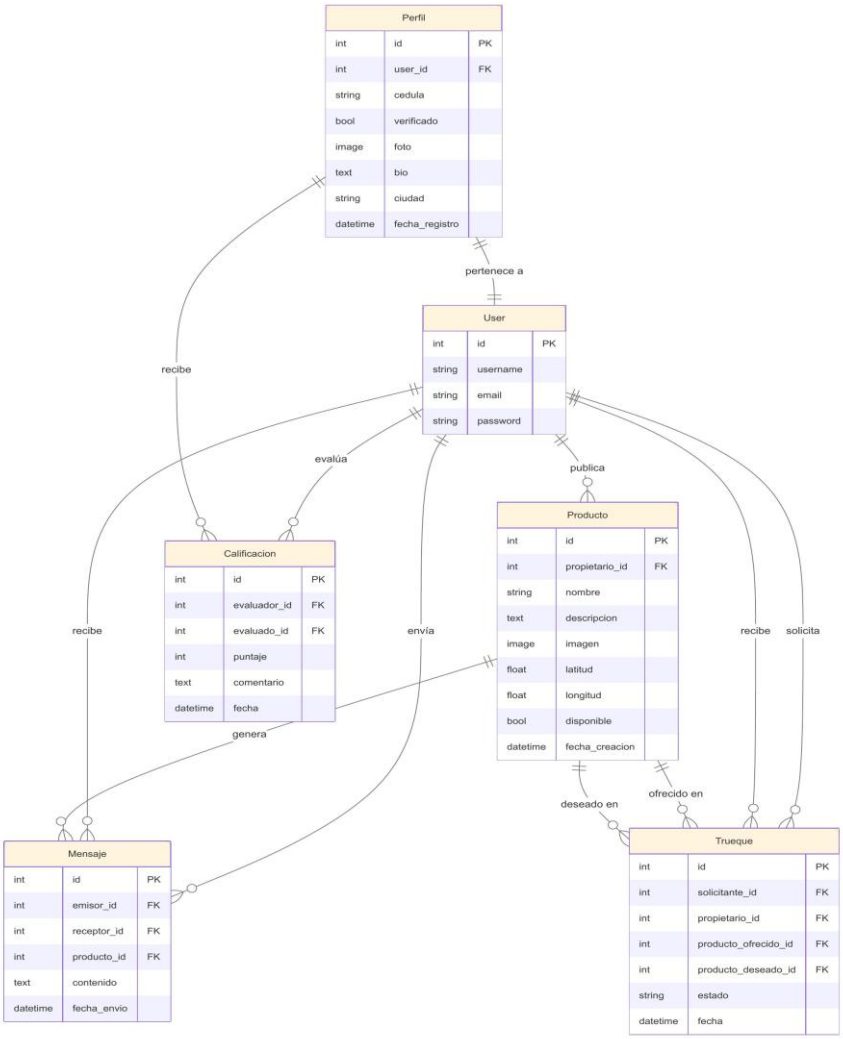


Figura 5. Diagrama de Base de Datos – Trueque Market

12. Estructuras Dinámicas Utilizadas

Durante el desarrollo del sistema se utilizaron listas, diccionarios, consultas dinámicas mediante Django ORM y estructuras relacionales PostgreSQL. Estas estructuras permitieron optimizar búsquedas, gestión de usuarios, consultas y administración de publicaciones.

13. Diagramas UML

A continuación se presentan los diagramas que describen el comportamiento, estructura y flujo del sistema Trueque Market.

13.1 Diagrama de Estados

El diagrama de estados representa el comportamiento dinámico del sistema y las transiciones entre autenticación, publicaciones, solicitudes y gestión de trueques.

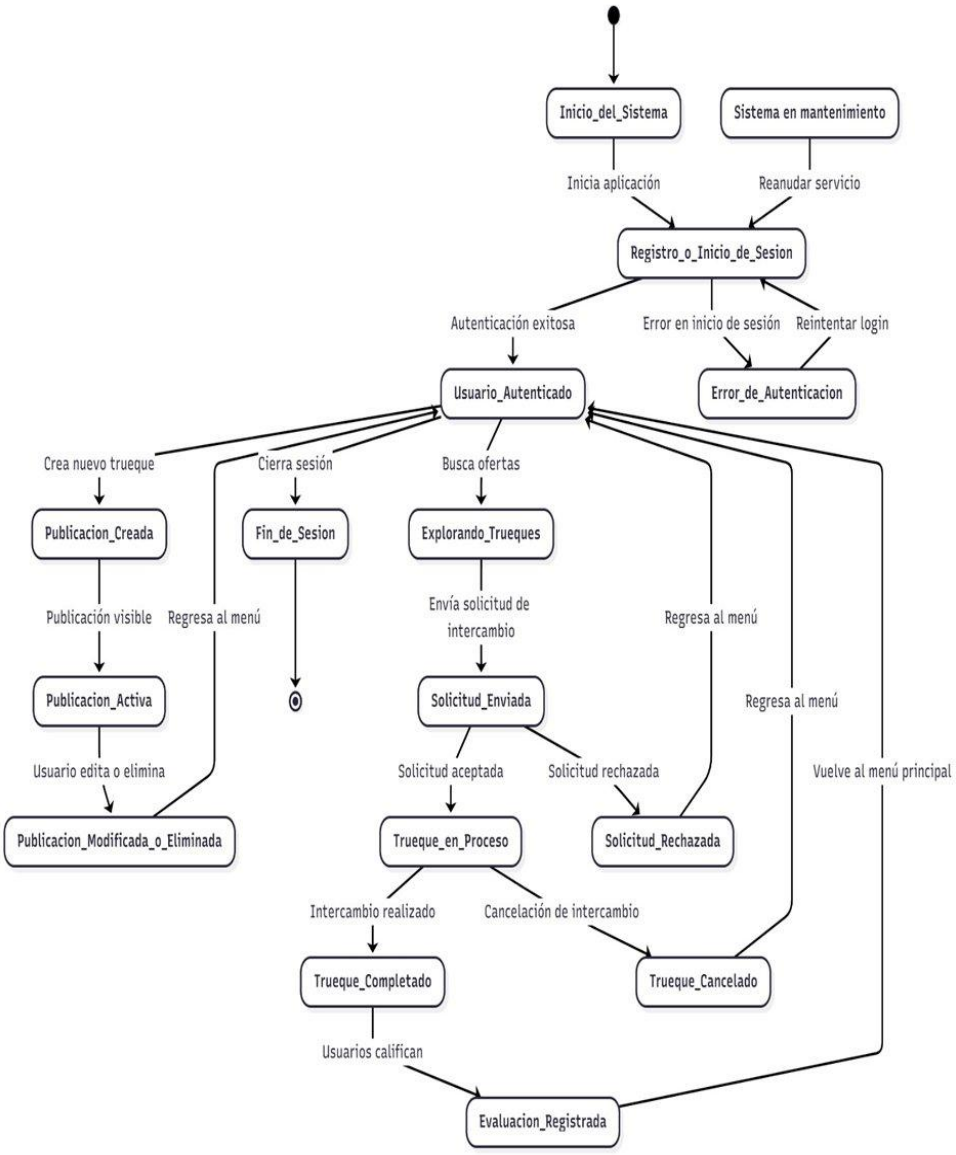


Figura 1. Diagrama de Estados – Trueque Market

13.2 Diagrama de Clases

El diagrama de clases representa las entidades principales del sistema, sus atributos, métodos y relaciones dentro del modelo orientado a objetos.

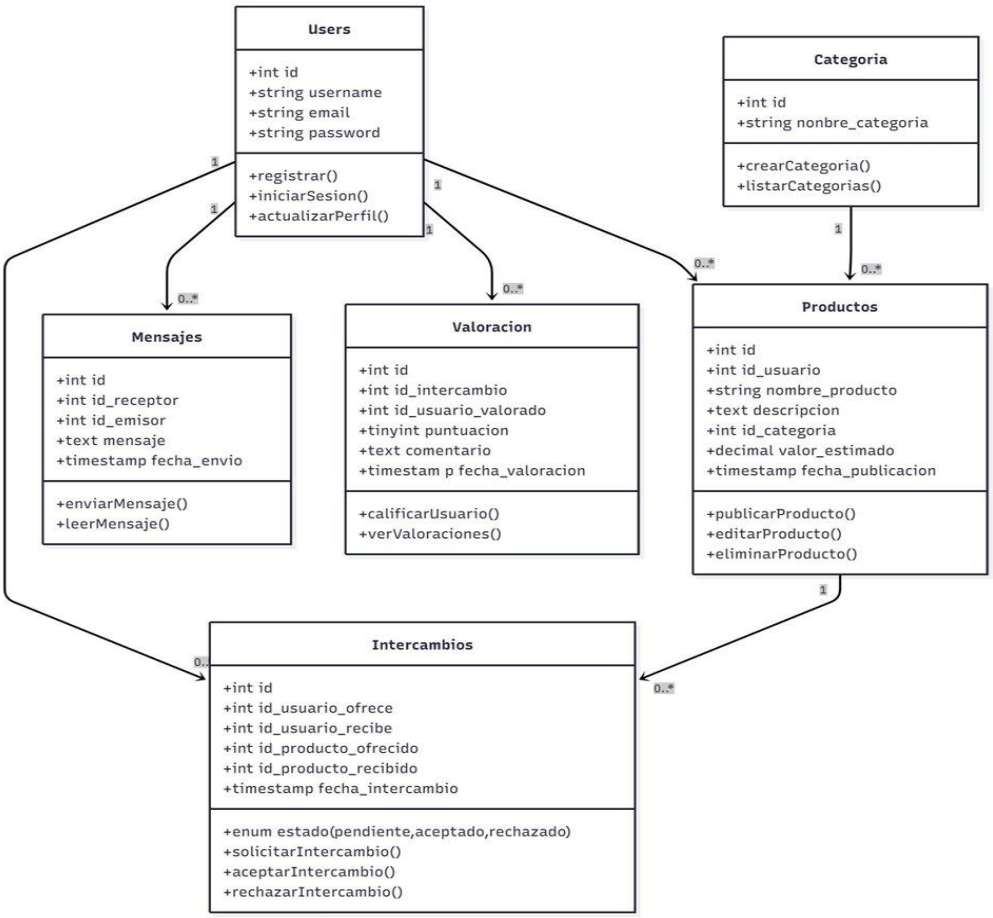


Figura 2. Diagrama de Clases – Trueque Market

13.3 Diagrama de Actividades

El diagrama de actividades describe el flujo de acciones que realiza un usuario dentro del sistema, desde el registro hasta la finalización de un trueque.

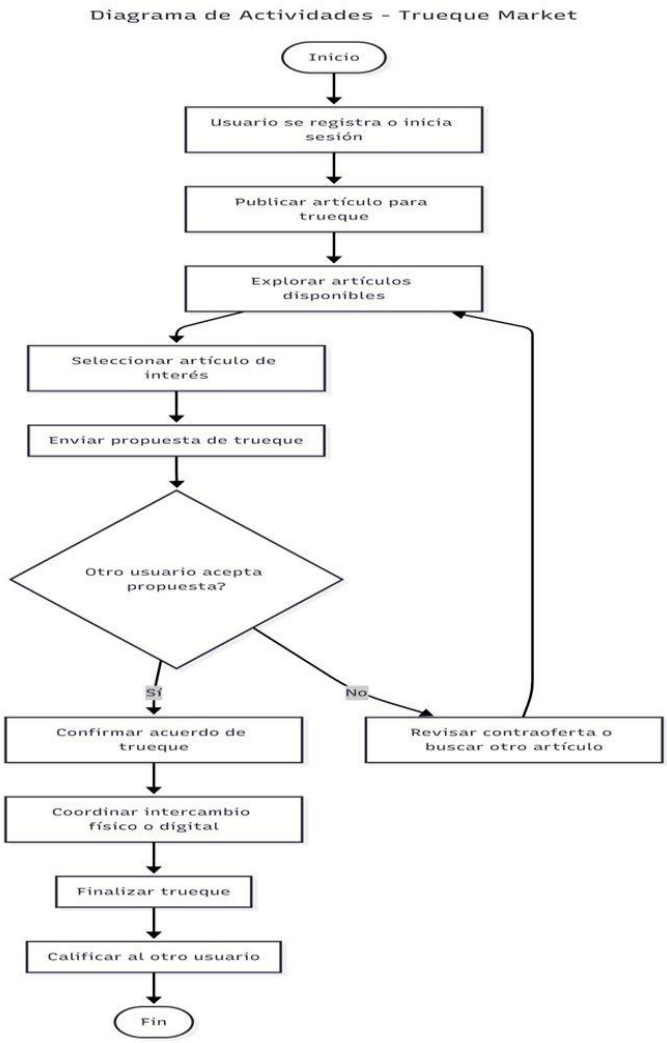


Figura 3. Diagrama de Actividades – Trueque Market

13.4 Diagrama de Casos de Uso

El diagrama de casos de uso representa las interacciones entre los actores del sistema (Usuario, Propietario y Administrador) y las funcionalidades disponibles.

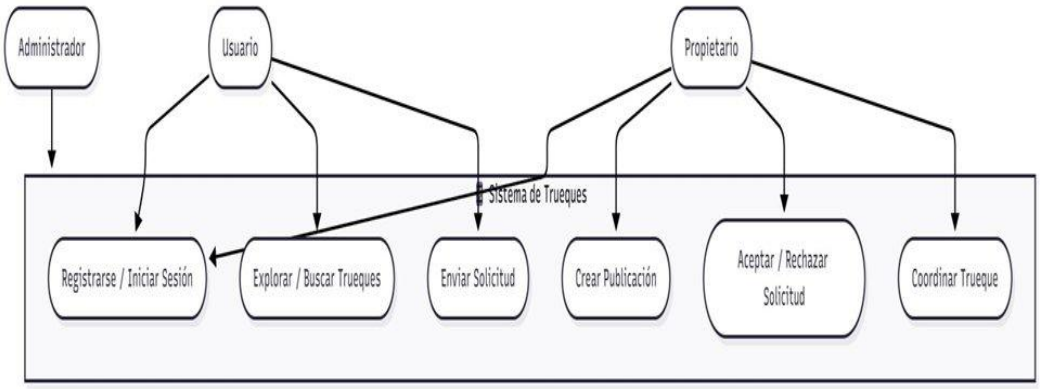


Figura 4. Diagrama de Casos de Uso – Trueque Market

14. Casos de Uso

14.1 Registro e Inicio de Sesión

Descripción: permite a los usuarios registrarse e iniciar sesión dentro de la plataforma.

Flujo principal

- El usuario ingresa sus credenciales.
- El sistema valida la información proporcionada.
- El usuario accede a la plataforma satisfactoriamente.

14.2 Crear Publicación

Descripción: permite registrar productos para intercambio.

Flujo principal

- El usuario selecciona la opción de crear publicación.
- Ingresa la información del producto.
- El sistema registra y publica la publicación.

14.3 Enviar Solicitud de Trueque

Descripción: permite solicitar un intercambio entre usuarios.

Flujo principal

- El usuario selecciona un producto de interés.
- Envía solicitud de intercambio al propietario.
- El propietario recibe y gestiona la solicitud.

14.4 Gestionar Trueque

Funciones disponibles: aceptar, rechazar, cancelar y completar trueques activos dentro del sistema.

14.5 Calificar Usuarios

Permite evaluar experiencias después de finalizar un intercambio, contribuyendo a la reputación de los usuarios en la plataforma.

15. Pruebas del Sistema

Se realizaron pruebas funcionales para validar los módulos principales del sistema:

- Autenticación y registro de usuarios.
- CRUD de publicaciones.
- Solicitudes de intercambio.
- Conexión con PostgreSQL.

- Estabilidad del despliegue en Render.

15.1 Resultados Obtenidos

- Correcto funcionamiento general del sistema.
- Persistencia adecuada de datos en PostgreSQL.
- Estabilidad del CRUD de publicaciones.
- Funcionamiento correcto de APIs integradas.
- Acceso remoto funcional mediante Render.

Durante la sustentación se realizará una demostración en vivo del funcionamiento de la plataforma, integrando Django, PostgreSQL y Render.

16. Rendimiento y Escalabilidad

El sistema fue diseñado considerando estabilidad, disponibilidad, rendimiento y escalabilidad futura. Los principales aspectos evaluados durante el desarrollo incluyeron:

- Velocidad de respuesta del sistema.
- Funcionamiento del CRUD.
- Estabilidad del despliegue.
- Conexión con PostgreSQL.
- Integración de APIs externas.

16.1 Resultados Observados

- Correcta persistencia de datos.
- Estabilidad en operaciones CRUD.
- Funcionamiento adecuado de Django.
- Integración estable con PostgreSQL.
- Disponibilidad continua mediante Render.

17. Conclusiones

El desarrollo de Trueque Market permitió aplicar de manera integral los conocimientos relacionados con desarrollo web, arquitectura de software, bases de datos, APIs, seguridad informática y metodologías ágiles.

La implementación de PostgreSQL permitió garantizar integridad y consistencia de los datos, mientras que Django facilitó la organización y seguridad del backend. El despliegue en Render permitió validar el funcionamiento real del sistema en un entorno web accesible remotamente.

Finalmente, el proyecto permitió desarrollar una solución tecnológica funcional orientada a la economía colaborativa mediante herramientas modernas de ingeniería de software, cumpliendo con los objetivos planteados al inicio del semestre.

Referencias

Django Software Foundation. (2024). Django documentation (versión 5.x). https://docs.djangoproject.com/

Mozilla Developer Network. (2024). HTML: HyperText Markup Language. https://developer.mozilla.org/es/docs/Web/HTML

Mozilla Developer Network. (2024). CSS: Cascading Style Sheets. https://developer.mozilla.org/es/docs/Web/CSS

OpenStreetMap Foundation. (2024). OpenStreetMap. https://www.openstreetmap.org/

PostgreSQL Global Development Group. (2024). PostgreSQL documentation. https://www.postgresql.org/docs/

Pusher Ltd. (2024). Pusher documentation. https://pusher.com/docs/

Python Software Foundation. (2024). Python documentation (versión 3.x). https://docs.python.org/3/

Render Services, Inc. (2024). Render documentation. https://render.com/docs

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum: The rules of the game. Scrum.org. https://scrumguides.org/scrum-guide.html